CommuniLend

Technical Documentation

Neighborhood tool library – ASP.NET Core MVC on .NET 10

Architecture · Domain model · Services · Web layer · Identity · Seed data · Configuration

Generated from the project source code, October 2026

Contents

Contents 2

1. Introduction 4

1.1 Purpose and scope of this document . . . 4

1.2 Key capabilities . . . 4

2. Solution Architecture 4

2.1 Request flow . . . 5

2.2 Design conventions . . . 5

3. Technology Stack 6

3.1 Prerequisites to build and run . . . 6

4. Domain Model 6

4.1 ApplicationUser . . . 7

4.2 Tool . . . 8

4.3 Category . . . 8

4.4 OpenLoan . . . 8

4.5 ClosedLoan . . . 8

4.6 Enumerations . . . 9

4.7 Validation and column constants . . . 9

5. Data Layer 9

5.1 CommuniLendDbContext . . . 9

5.2 Relationships and delete behavior . . . 9

5.3 Migrations . . . 10

5.4 Seed data . . . 10

6. Services Layer 10

6.1 ToolsService (IToolsService) . . . 11

6.2 LoansService (ILoansService) . . . 11

6.3 UsersService (IUsersService) . . . 12

6.4 Availability status (ToolQueryExtensions.WithStatus) . . . 12

6.5 Image storage (IImageStorageService / LocalImageStorageService) . . . 12

6.6 Helpers . . . 12

7. Web Layer 13

7.1 Program.cs – startup and pipeline . . . 13

7.2 BaseController . . . 13

7.3 Route and action reference . . . 14

7.4 Controller notes . . . 14

8. View Models **15**

8.1 Validation rules . . . 16

9. Views and User Interface **16**

9.1 Layout . . . 16

9.2 View inventory . . . 16

9.3 Status badge . . . 17

9.4 Typical user journeys . . . 17

10. Identity and Registration **17**

10.1 Two-step registration . . . 17

10.2 RegistrationTokenProtector . . . 17

10.3 Login and logout . . . 17

10.4 Authorization model . . . 18

11. Configuration and Operations **18**

11.1 Configuration files . . . 18

11.2 Identity password options . . . 18

11.3 Build, migrate, run . . . 18

11.4 Static assets and uploads . . . 18

12. Seed Data Reference **19**

12.1 Members . . . 19

12.2 Categories . . . 19

12.3 Open loans (7) . . . 19

12.4 Closed loans (9) . . . 20

12.5 Tools (50) . . . 21

13. Observations on Current Behavior **22**

Appendix A. Source File Inventory **23**

1. Introduction

CommuniLend is a neighborhood tool library. Members list the tools they own, browse tools that neighbors share, borrow them for a chosen period, and keep a history of what they lent and borrowed. The solution is a layered ASP.NET Core MVC application targeting **.NET 10**, persisting data in **SQL Server** through **Entity Framework Core 10** and authenticating users with **ASP.NET Core Identity**.

1.1 Purpose and scope of this document

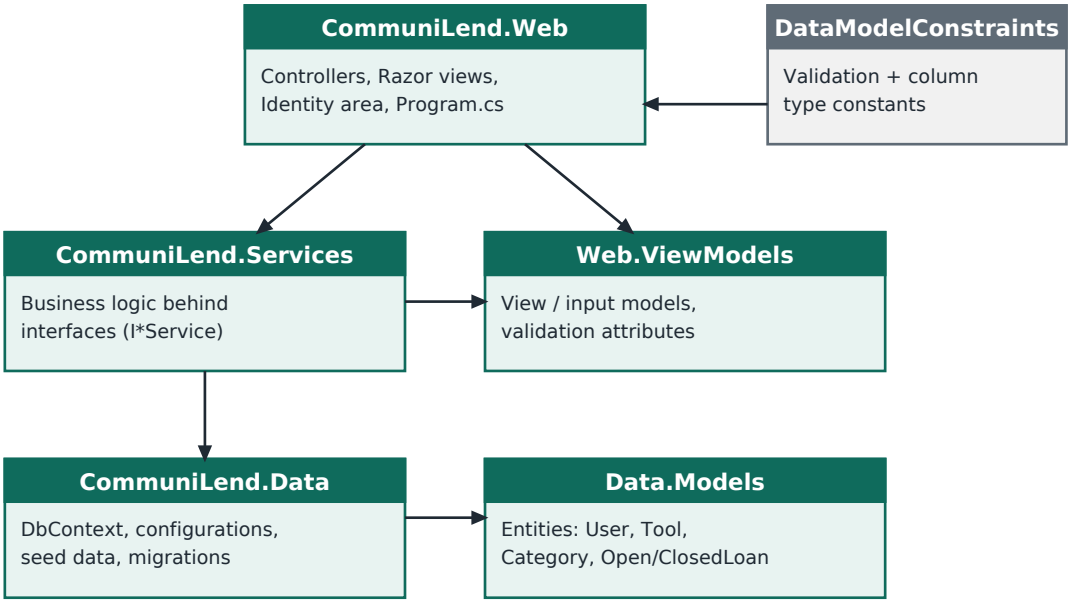
This document describes the structure and behavior of the code base as found in the repository: solution layout, domain model, database mapping, services, controllers, view models, views, the customized registration flow, configuration and seed data. It also lists a few observations about the current behavior (chapter 13). It is not a user manual; the project README covers installation steps.

1.2 Key capabilities

- Registration (two steps), login and logout with ASP.NET Core Identity; members have a full name, phone number and optional address.
- Tool catalog of other active members' tools with category, owner, photo and availability status; search by name and filter by category.
- Full CRUD for the member's own tools, including photo upload (JPG/PNG, max 50 KB).
- Loans: borrow an available tool for a future period; the lender marks it returned, which moves the loan to history.
- Dashboard and public member profiles showing owned, provided and borrowed tools and loan history.
- Profile editing (including an active flag) and account deletion with safeguards.
- Client- and server-side validation; seed data (3 categories, 7 members, 50 tools, 7 open and 9 closed loans); database column comments.

2. Solution Architecture

The solution file **CommuniLend.slnx** contains six projects. Dependencies point downward; the web project is the only executable.



Project	Type	Responsibility	References
CommuniLend.Web	ASP.NET Core Web (net10.0)	Controllers, Razor views, Identity area, static files, `Program.cs`, configuration	Services, Web.ViewModels
CommuniLend.Web.ViewModels	Class library	View and input models, enum `ToolAvailabilityStatus`, upload validation attributes	Data.Models.Constraints (constants)
CommuniLend.Services	Class library	Business logic behind `I*Service` interfaces; image storage; query helpers	Data, Web.ViewModels (Data.Models transitively)
CommuniLend.Data	Class library	`CommuniLendDbContext`, entity configurations, seed data, EF migrations	Data.Models
CommuniLend.Data.Models	Class library	Entities and enum `ToolCondition`	Data.Models.Constraints
CommuniLend.DataModelConstraints	Class library	Shared constants: `ValidationConstants`, `ColumnTypes`	–

The reference column lists the direct `ProjectReference` entries from each .csproj file.

2.1 Request flow

A typical request travels: browser → ASP.NET Core pipeline (HTTPS redirection, routing, authentication, authorization) → MVC controller (derived from `BaseController`, which carries `[Authorize]`) → service interface → EF Core `CommuniLendDbContext` → SQL Server. Services return view models, which the controller passes to a Razor view. Controllers contain no queries; they translate service results to views, redirects or HTTP status codes.

2.2 Design conventions

- **Service interfaces** in `CommuniLend.Services.Contracts`; implementations registered as scoped services.
- **Read queries** use `AsNoTracking()` and `AsSplitQuery()`; results are projected to view models in the service.

- **Ownership checks live in the services**: methods receive the current `userId` and filter by it, so a member cannot act on another member's data.
- **Dates** are `DateOnly` in the model and are formatted as `dd MMMM yyyy` (invariant culture) for the views.
- **Money** is `decimal(8,2)` in EUR and is formatted with the invariant culture (`0.00`).
- **Required members** use C# `required` and nullable reference types are enabled.

3. Technology Stack

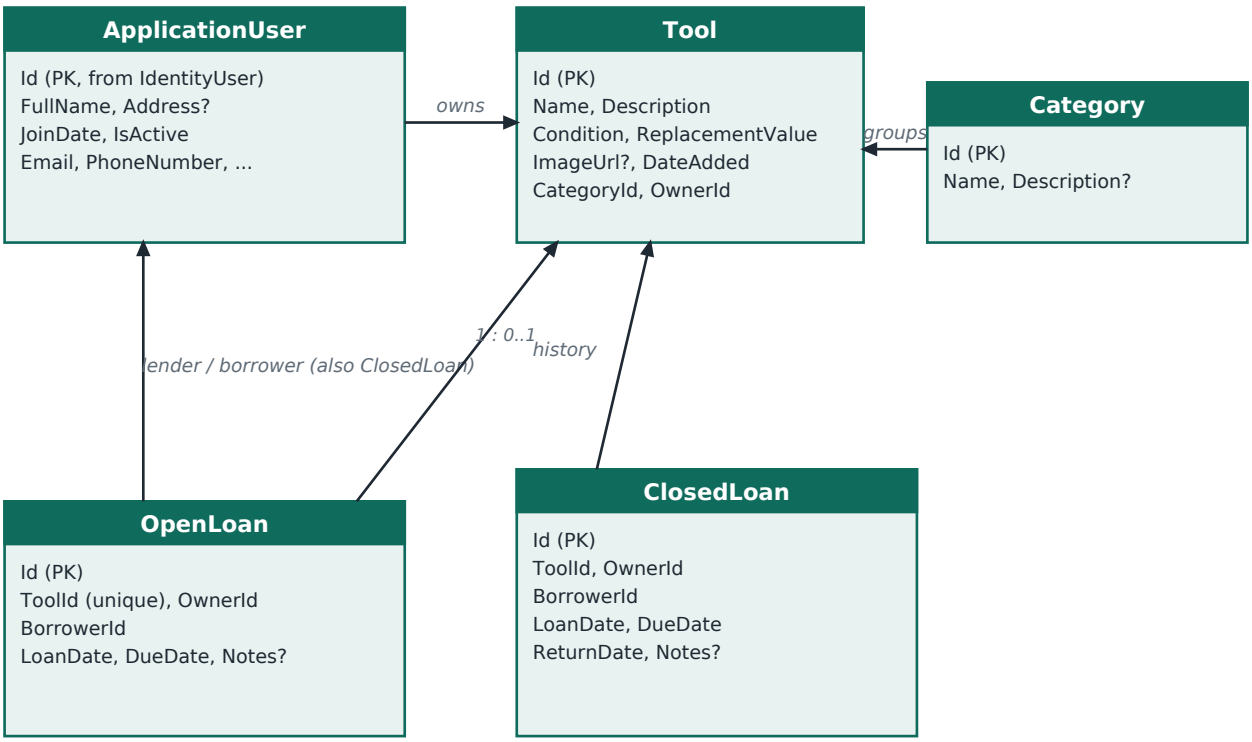
Technology	Version	Purpose
.NET / ASP.NET Core MVC	10.0 (net10.0)	Web framework, Razor views
Microsoft.EntityFrameworkCore.SqlServer	10.0.13	ORM and SQL Server provider
Microsoft.EntityFrameworkCore.Design / Tools	10.0.13	Migrations tooling (design-time only)
Microsoft.AspNetCore.Identity.EntityFrameworkCore	10.0.13	Identity stores on EF Core
Microsoft.AspNetCore.Identity.UI	10.0.13	Default Identity Razor Pages (customized pages in the Identity area)
Microsoft.AspNetCore.Diagnostics.EntityFrameworkCore	10.0.13	Database developer exception page and migrations endpoint
System.Security.Cryptography.Pkcs	10.0.13	Explicit package reference (security patch level)
Bootstrap	5	Layout and components (bundled in wwwroot/lib)
jQuery, jQuery Validation, Unobtrusive Validation	bundled	Client-side validation and scripts

3.1 Prerequisites to build and run

- **.NET 10 SDK**
- SQL Server (LocalDB, Express or full instance)
- EF Core CLI tools (`dotnet tool install --global dotnet-ef`)

4. Domain Model

Entities live in `CommuniLend.Data.Models`. Every entity and column carries a `[Comment]` attribute, which EF Core applies as a SQL Server extended property (see chapter 5). Navigation properties are `virtual`.



All foreign keys use ON DELETE RESTRICT. Arrows point from the dependent entity to the principal entity.

4.1 ApplicationUser

Extends `IdentityUser` (string key, 450 characters). Represents a registered member.

Property	Type	Constraints	Description
`FullName`	string	Required, max 100	Full name of the member
`Address`	string?	Max 200	Optional home address
`JoinDate`	DateOnly	Required	Date the member joined
`IsActive`	bool	Required	Inactive members' tools are hidden (status Locked) and they cannot borrow
`PhoneNumber`	string? (inherited)	Max 15 (configured)	Constraint added in ApplicationUserConfiguration
`OwnedTools`	ICollection<Tool>	–	Tools owned by the member
`ProvidedTools / BorrowedTools`	ICollection<OpenLoan>	–	Active loans as lender / borrower
`ReturnedProvidedTools / ReturnedBorrowedTools`	ICollection<ClosedLoan>	–	History as lender / borrower

4.2 Tool

Property	Type	Constraints	Description
`Id`	int	PK	Primary key
`Name`	string	Required, max 100 (form: 3–100)	Name of the tool
`Description`	string	Required, max 500 (form: 3–500)	Description of the tool
`Condition`	ToolCondition	Required	Enum: New, Good, Fair, Worn (stored as int)
`ReplacementValue`	decimal	Required, decimal(8,2)	Replacement value in EUR
`ImageUrl`	string?	Max 512, indexed	Relative URL of the image, e.g. /images/tools/level.jpg
`DateAdded`	DateOnly	Required	Date added to the library
`CategoryId / Category`	int / Category	Required, FK	Category of the tool
`OwnerId / Owner`	string / ApplicationUser	Required, FK (max 450)	Owner of the tool
`OpenLoan`	OpenLoan?	0..1	Current active loan, if any
`ClosedLoans`	ICollection<ClosedLoan>	–	Loan history of the tool

4.3 Category

Property	Type	Constraints	Description
`Id`	int	PK	Primary key
`Name`	string	Required, max 100	Name of the category
`Description`	string?	Max 500	Optional description
`Tools`	ICollection<Tool>	–	Tools in the category

4.4 OpenLoan

An active loan that has not been returned yet. There is at most one open loan per tool (unique index on `ToolId`).

Property	Type	Constraints	Description
`Id`	int	PK	Primary key
`ToolId / Tool`	int / Tool	Required, FK, unique	Loaned tool
`OwnerId / Owner`	string / ApplicationUser	Required, FK	Lender (owner of the tool)
`BorrowerId / Borrower`	string / ApplicationUser	Required, FK	Borrower
`LoanDate`	DateOnly	Required	Date the tool is handed over
`DueDate`	DateOnly	Required	Date the tool is expected back
`Notes`	string?	Max 1000	Optional notes

4.5 ClosedLoan

History record created when a tool is returned. Same columns as `OpenLoan` plus `ReturnDate` (required, the actual return date). `ToolId` is *not* unique, so a tool can have many closed loans.

4.6 Enumerations

Enum	Values	Notes
`ToolCondition`	New (0), Good (1), Fair (2), Worn (3)	Persisted on Tool.Condition; shown in forms through ConditionOptionModel
`ToolAvailabilityStatus`	OverDue, OnLoan, Available, Locked	View-model enum, computed at query time (section 6.4); never stored

4.7 Validation and column constants

Constant	Value
User.FullNameMaxLength / EmailMaxLength / PhoneNumberMaxLength / AddressMaxLength	100 / 254 / 15 / 200
User.ApplicationUserIdMaxLength	450
Tool.NameMinLength / NameMaxLength	3 / 100
Tool.DescriptionMinLength / DescriptionMaxLength	3 / 500
Tool.ImageUrlMaxLength	512
Tool.PictureMaxFileSize	50 × 1024 bytes (50 KB)
Category.NameMaxLength / DescriptionMaxLength	100 / 500
Loan.NotesMaxLength	1000
ColumnTypes.Tool.Decimal	decimal(8,2)

5. Data Layer

5.1 CommuniLendDbContext

`CommuniLendDbContext` derives from `IdentityDbContext<ApplicationUser>` (so the standard Identity tables exist alongside the domain tables) and exposes four sets: `Categories`, `Tools`, `ClosedLoans` and `OpenLoans`. `OnModelCreating` calls the base implementation and then applies one `IEntityTypeConfiguration` per entity.

5.2 Relationships and delete behavior

Relationship	Configured in	On delete
Tool → Category (many-to-one)	ToolConfiguration	Restrict
Tool → Owner (many-to-one, ApplicationUser.OwnedTools)	ToolConfiguration	Restrict
OpenLoan → Tool (one-to-one, unique index on ToolId)	OpenLoanConfiguration	Restrict
OpenLoan → Owner (ApplicationUser.ProvidedTools)	OpenLoanConfiguration	Restrict
OpenLoan → Borrower (ApplicationUser.BorrowedTools)	OpenLoanConfiguration	Restrict
ClosedLoan → Tool (many-to-one, Tool.ClosedLoans)	ClosedLoanConfiguration	Restrict
ClosedLoan → Owner (ApplicationUser.ReturnedProvidedTools)	ClosedLoanConfiguration	Restrict
ClosedLoan → Borrower (ApplicationUser.ReturnedBorrowedTools)	ClosedLoanConfiguration	Restrict

Because all foreign keys restrict deletion, removing a tool or a user requires deleting dependent rows first. The services do this explicitly inside a transaction (sections 6.2 and 6.3). Additional mapping: `Tool.ImageUrl` has an index; `ApplicationUser.PhoneNumber` is limited to 15 characters.

5.3 Migrations

Migration	Timestamp (UTC id)	Purpose (from name and content)
`MigrateToIdentity`	2026-10-18 09:12:29	Creates the Identity tables and domain tables on ASP.NET Core Identity; includes initial seed data
`AddedRestrictionForImageUrl`	2026-10-20 16:39:50	Adds restrictions to the ImageUrl column and alters several columns
`AddedComments`	2026-10-21 20:03:02	Applies [Comment] descriptions to tables and columns (altering many columns)
`ImageUrlAsIndex`	2026-10-22 10:25:46	Adds the index on Tool.ImageUrl

The model snapshot is `CommuniLendDbContextModelSnapshot.cs`. Apply migrations with:

```
dotnet ef database update --project CommuniLend.Data --startup-project CommuniLend.Web
```

5.4 Seed data

Seed data is supplied through `HasData` in the entity configurations, so it is created by the migrations. Fixed keys and dates keep migrations deterministic:

- `ReferenceSeedDate.ReferenceDate` = **28 September 2026**; all seeded dates are offsets from it (for example `ReferenceDate.AddDays(-14)`).
- `SeedUserIds` holds seven fixed GUID strings, so loans and tools can reference the seeded members.
- Seeded users share one precomputed password hash for the password **Password123!** (`ApplicationUserConfiguration.SeedPassword`); all are active with confirmed e-mail and phone.
- Seed totals: 3 categories, 7 members, 50 tools, 7 open loans, 9 closed loans (complete listing in chapter 12).

6. Services Layer

All services are registered with scoped lifetime in `Program.cs`. They depend on `CommuniLendDbContext`, `UserManager<ApplicationUser>` where needed, and `IImageStorageService`.

6.1 ToolsService (IToolsService)

Method	Behavior
`GetAllAsync(userId, search, categoryId)`	Catalog query: tools of *other* members whose owner is active; optional name filter (trimmed, `Contains`) and category filter. Computes availability status, orders by status descending, then category name, then owner name. Also returns categories ordered by name and echoes the search values.
`GetDetailsAsync(toolId)`	Full tool details with category, owner and, if loaned, the borrower and dates. Returns null if not found.
`GetDeleteDetailsAsync(userId, toolId)`	Delete-confirmation model for a tool owned by the user; `IsDeletable` is true only when there is no open loan.
`DeleteAsync(userId, toolId)`	In an execution-strategy transaction: bulk-deletes the tool's closed loans, then deletes the tool (only if owned by the user and not on loan). Rolls back unless exactly one tool row was deleted. After success, deletes the image file via IImageStorageService.
`GetAddModelAsync()`	Form model with all categories and all `ToolCondition` values.
`ValidateInputDataAsync(model)`	Async validation: category must exist; condition id must be a defined enum value. Yields `ValidationResults`.
`AddAsync(userId, model)`	Creates the tool: parses the price with the invariant culture, sets `DateAdded` to today (UTC), saves the uploaded image (if any) and stores its URL.
`GetEditModelAsync(userId, toolId)`	Edit form model; null unless the user owns the tool and it has no open loan.
`EditAsync(userId, toolId, model)`	Updates name, description, condition, price and category; replaces the image URL only when a new file is uploaded; after saving, the replaced image file is deleted through IImageStorageService (the placeholder and images still used by another tool are kept). Returns false if the tool is not owned, or is on loan.

6.2 LoansService (ILoansService)

Method	Behavior
`GetAllAsync(userId)`	Returns two lists: tools the user *provides* that are on loan (with borrower) and tools the user *borrows* (with owner). Each is sorted by status, due date descending, counterpart name, tool name.
`ReturnAsync(userId, openLoanId)`	Only the lender (owner) can return. Creates a `ClosedLoan` copying the loan fields with `ReturnDate` = today (UTC), removes the `OpenLoan`, saves. Returns false if the loan does not exist or the user is not the owner.
`GetAddModelAsync(userId, toolId)`	Borrow form model. Null if the user is missing or inactive, or the tool is missing, owned by the user, has an inactive owner, or already has an open loan.
`CreateAsync(userId, toolId, model)`	Re-checks the same conditions, then creates an `OpenLoan` with the parsed loan/due dates and notes. Returns false when a condition fails.

6.3 UsersService (IUsersService)

Method	Behavior
`GetDetailsAsync(userId)`	Public profile: user data plus five lists – owned tools, provided (on loan), borrowed, returned-provided and returned-borrowed – each with its count.
`GetDashboardDetailsAsync(userId)`	Same model for the signed-in user's dashboard.
`EditCurrentDetailsAsync(userId)`	Pre-filled edit model (name, e-mail, phone, address, active flag).
`UpdateUserDetailsAsync(userId, model)`	Copies the fields to the user and sets `UserName` to the new e-mail (the e-mail is the login name), then calls `UserManager.UpdateAsync`, which refreshes the normalized values and rejects a duplicate user name; returns whether it succeeded.
`GetDeleteModelAsync(userId)`	Returns `HasActiveLoans` (any open loan as owner or borrower).
`DeleteAsync(userId)`	Refuses while open loans exist. Otherwise, in a transaction, deletes closed loans where the user is owner or borrower, deletes the user's tools, then deletes the user through UserManager; rolls back on failure. Afterwards deletes the now-unused image files.

6.4 Availability status (ToolQueryExtensions.WithStatus)

The status is computed in the database query by projecting each `Tool` to a `ToolWithStatus(Tool, Status)` record. “Today” is the UTC date. The rules, in order:

Condition	Status	Badge
No open loan, owner is active	Available	green
No open loan, owner is inactive	Locked	dark
Open loan with DueDate ≥ today	OnLoan	yellow
Open loan with DueDate < today	OverDue	red

Sorting on the status uses the enum order `OverDue, OnLoan, Available, Locked` (numeric 0–3). Lists of loans sort ascending (overdue first); the catalog sorts descending (available first).

6.5 Image storage (IImageStorageService / LocalImageStorageService)

Member	Behavior
`Storage location`	`{WebRootPath}/images/tools/`; URLs are returned as `/images/tools/<file>`.
`SaveAsync(file)`	Returns null for no file. Uses `Path.GetFileName` of the uploaded name; while a file with that name exists, prefixes a new GUID (`N` format). Writes with `FileMode.CreateNew` and a 50 KB buffer.
`DeleteAsync(url)`	Skips null, skips the placeholder `no-image.jpg`, skips images still referenced by any tool, then deletes the file; IO errors are swallowed.
`DeleteAsync(urls)`	Deletes a collection one by one.

6.6 Helpers

- `DateOnlyExtensions.ToFormattedString` – formats with `dd MMMM yyyy`, invariant culture.
- `ToolQueryExtensions.WithStatus` – see 6.4. Both helpers are `internal` to the services project.

7. Web Layer

7.1 Program.cs – startup and pipeline

- Reads `ConnectionStrings:DefaultConnection`; throws `InvalidOperationException` if missing. Registers the DbContext with `UseSqlServer` and the database developer exception filter.
- `AddDefaultIdentity<ApplicationUser>` with `RequireConfirmedAccount = false`; password options are bound from the `Identity:Password` configuration section; stores on `CommuniLendDbContext`.
- `AddControllersWithViews` with a global `AutoValidateAntiforgeryTokenAttribute` filter (CSRF protection on all unsafe HTTP methods).
- Service registrations (scoped): `IToolsService → ToolsService`, `IUsersService → UsersService`, `ILoansService → LoansService`, `IImageStorageService → LocalImageStorageService`.
- Development: migrations endpoint and developer exception page. Otherwise HSTS.
- Pipeline order: HTTPS redirection → routing → authentication → authorization → static assets → default MVC route → Razor Pages (Identity UI).

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Tools}/{action=Index}/{id?}")
    .WithStaticAssets();
app.MapRazorPages().WithStaticAssets();
```

The tool catalog is therefore the home page (`/ → Tools/Index`).

7.2 BaseController

Abstract base for all MVC controllers. Marked `[Authorize]`, so every MVC action requires a signed-in user (unauthenticated visitors are redirected to the Identity login page). Provides `GetUserId()` (via `UserManager.GetUserId`, empty string if absent) and `IsUserAuthenticated()`.

7.3 Route and action reference

Method	URL	Action	Behavior / outcomes
GET	/Tools?search=&category=	ToolsController.Index	Catalog with search and category filter
GET	/Tools/Details/{id}	Details	Tool page; redirects to catalog if not found
GET	/Tools/Add	Add	Add-tool form
POST	/Tools/Add	Add	Request size limit 50 KB; runs service validation; success → redirect to Dashboard; failure → 400
GET	/Tools/Edit/{id}	Edit	404 if not owner or tool is on loan
POST	/Tools/Edit/{id}	Edit	Validates, updates; success → Dashboard; failure → 400
GET	/Tools/DeleteDetails/{id}	DeleteDetails	Confirmation page; redirects to Dashboard if not owned
POST	/Tools/DeleteConfirm/{id}	DeleteConfirm	Success → Dashboard; failure → 409 Conflict
GET	/Loans	Index	Provided and borrowed active loans
GET	/Loans/Create/{id}	Create	Borrow form for tool {id}; redirects to tool details if not borrowable
POST	/Loans/Create/{id}	Create	Invalid model → redisplay; service failure → 400; success → Loans
POST	/Loans/Return/{id}	Return	{id} = open loan id; lender only; always redirects to Loans
GET	/Users/Dashboard	Dashboard	Signed-in member's profile, tools and loan history; 404 if user missing
GET	/Users/Details/{id}	Details	Another member's profile; own id → Dashboard; unknown id → Dashboard
GET / POST	/Users/Edit	Edit	Profile edit; success → sign-in cookie refreshed (so the navbar shows the new e-mail) and redirect to Dashboard; failure → 400
GET	/Users/Delete	Delete	Account-deletion page showing whether open loans block it
POST	/Users/DeleteConfirmed	DeleteConfirmed	Success → sign out, redirect to catalog; blocked → back to Delete page
GET / POST	/Identity/Account/Register	RegisterModel	Step 1 of registration (credentials)
GET / POST	/Identity/Account/RegisterDetails	RegisterDetailsModel	Step 2 of registration (profile); creates the user
GET / POST	/Identity/Account/Login, Logout, AccessDenied	Login/Logout/AccessDenied	Scaffolded Identity pages (Logout is POST)

7.4 Controller notes

- The route parameter is bound as `[FromRoute(Name = "id")]` and mapped to descriptive parameter names (`toolId`, `openLoanId`, `userId`).
- The catalog binds the category filter from the query-string key `category` to `categoryId`.

- `ToolsController.Add` and `Edit` merge the service's async `ValidationResults` into `ModelState` before checking validity.
- `ToolsController.Add` carries `[RequestSizeLimit]` and `[RequestFormLimits]` set to `PictureMaxFileSize` (50 KB).
- Redirects use `nameof(...)` with controller names, avoiding magic strings.

8. View Models

View models are plain classes in `CommuniLend.Web.ViewModels`. Dates and amounts arrive already formatted as strings; availability is carried as the `ToolAvailabilityStatus` enum.

Folder / class	Used by	Contents
`Category/CategoryViewModel`	Catalog, tool forms	Id, Name
`Tool/IndexViewModel`, `ToolIndexViewModel`	Catalog	Tools (id, name, image, status, owner, category), Categories, SearchString, CategoryId
`Tool/DetailsViewModel`	Tool details	All tool fields, owner, optional borrower, loan and due dates, OpenLoanId, status
`Tool/ToolFormInputViewModel`	Add / Edit	ToolName, CategoryId, Description, ConditionId, ReplacementValue (string), ToolUploadImage, option lists, ToolImageUrl
`Tool/ToolViewModel`	Borrow page	Read-only tool summary embedded in the loan form
`Tool/DeleteViewModel`	Tool delete	Tool summary plus IsDeletable
`Tool/ImageUploadViewModel`	Add / Edit	ToolPictureUploadViewModel and three validation attributes (below)
`Loan/CreateInputViewModel`	Borrow form	ToolViewModel, LoanDate, DueDate (strings), Notes; implements IValidatableObject
`Loan/ActiveLoanIndexViewModel`	Loans page	ProvidedToolViewModel and BorrowedToolViewModel lists
`User/DetailsViewModel (+ nested row models)`	Dashboard, profile	UserData, five lists with counts
`User/UserViewModel`	Profile	Id, name, e-mail, phone, address, member-since, active flag
`User/EditInputViewModel`	Profile edit	FullName, Email, PhoneNumber, Address, IsActive
`User/DeleteViewModel`	Account delete	HasActiveLoans

8.1 Validation rules

Field	Rules
ToolFormInputViewModel.ToolName	Required, 3–100 characters
ToolFormInputViewModel.Description	Required, 3–500 characters
ToolFormInputViewModel.ReplacementValue	Required; regular expression `^\d{1,6}([.]\d{1,2})?$` – message "Price must be between 0.01 and 999999.99."
ToolFormInputViewModel.CategoryId / ConditionId	Required; existence checked server-side by ToolsService.ValidateInputDataAsync
ToolPictureUploadViewModel.File	`[MaximumFileSizeInBytes(50 KB)]`, `[AllowedExtensions(.jpg, .jpeg, .png)]` (case-insensitive), `[ImageFile]`
CreateInputViewModel.LoanDate / DueDate	Required; must parse; loan date must be after today (UTC); due date must be after loan date. Notes max 1000.
EditInputViewModel	FullName required max 100; Email required, valid, max 254; PhoneNumber required, valid, max 15; Address max 200
Register page	Email required and valid; password required, max 100 characters; the minimum length and complexity come from the Identity password rules in configuration (no fixed minimum on the page); confirmation must match
RegisterDetails page	FullName required max 100; PhoneNumber required, valid, max 15; Address optional max 200

Custom attributes. `ImageFileAttribute` validates the file signature (magic bytes): JPEG starts with `FF D8 FF`, PNG with `89 50 4E 47`, so renaming another file type to .jpg is rejected. `AllowedExtensionsAttribute` checks the extension. `MaximumFileSizeInBytesAttribute` checks `IFormFile.Length`.

9. Views and User Interface

9.1 Layout

`Views/Shared/_Layout.cshtml` renders the Bootstrap 5 navbar (brand, **Tools**, **Loans**, **Dashboard** links with the active controller highlighted, and `_LoginPartial`), the main container with `RenderBody`, a footer, and the scripts (jQuery, Bootstrap bundle, `site.js`). Pages may supply `Styles` and `Scripts` sections; forms include `_ValidationScriptsPartial` for jQuery Validation / Unobtrusive.

9.2 View inventory

Area	Views and partials
Tools	Index (catalog with search/filter form), Details, Add, Edit, DeleteDetails, _ToolFormPartial (shared by Add/Edit), _ToolIndexItemPartial (catalog card)
Loans	Index, Create (borrow), _ProvidedToolsPartial, _BorrowedToolsPartial
Users	Dashboard, Details, Edit, Delete, _UserInfoPartial, _OwnedToolsPartial, _ProvidedToolsPartial, _BorrowedToolsPartial, _PastProvidedToolsPartial, _PastBorrowedToolsPartial
Shared	_Layout (+ scoped CSS), _LoginPartial, _ValidationScriptsPartial, _ToolStatusPartial (status badge), _ToolImgePartial (image with fallback)
Identity area	Login, Logout, Register, RegisterDetails, AccessDenied, Error, _StatusMessage

9.3 Status badge

`_ToolStatusPartial` maps each `ToolAvailabilityStatus` to a label and Bootstrap class: Available → green, On Loan → yellow, Locked → dark, Over Due → red (unknown values fall back to grey "Unknown").

9.4 Typical user journeys

Journey	Steps
Borrow a tool	Catalog → open tool → Borrow → choose loan date (future) and due date, optional notes → submit → appears on Loans (borrowed) and the tool shows On Loan.
Return a tool	Lender opens Loans → provided tools → Return → loan moves to history for both members; tool is Available again.
Share a tool	Dashboard / Tools → Add → fill the form with optional photo → tool appears in other members' catalogs.
Leave the library	Dashboard → Delete account → blocked while open loans exist; otherwise removes history, tools, photos and the account, then signs out.

10. Identity and Registration

The application uses ASP.NET Core Identity with the scaffolded UI in `Areas/Identity/Pages/Account`. Sign-in does not require confirmed accounts. Password rules come from configuration (chapter 11).

10.1 Two-step registration

Registration is split so that nothing is written to the database until all profile data has been entered.

Step	Page	What happens
1	Register	Validates e-mail and password. Rejects an e-mail that already exists and runs the configured Identity password validators against a probe user. Nothing is saved. The credentials are encrypted into a token and placed in `TempData`; redirects to step 2.
2	RegisterDetails	Reads the token (via `TempData.Peek`, so a page refresh works). If it is missing, expired or tampered with, redirects back to step 1. Collects full name, phone and optional address. On POST creates the `ApplicationUser` (JoinDate = today UTC, IsActive = true, e-mail/phone confirmed, lockout disabled), signs the user in and redirects.

10.2 RegistrationTokenProtector

Uses ASP.NET Core Data Protection with the purpose string `CommuniLend.Registration.v1` and a **time-limited protector with a 30-minute lifetime**. The pending e-mail and password are serialized to JSON and protected, so the password is neither stored in the database nor sent to the browser in clear text. `Unprotect` returns null for empty, tampered, malformed or expired tokens.

10.3 Login and logout

Login calls `PasswordSignInAsync(Input.Email, Input.Password, RememberMe, lockoutOnFailure: false)`. Logout is a POST that calls `SignOutAsync`. Deleting an account also signs the user out.

10.4 Authorization model

There are no roles. Authorization is binary (signed in or not, through `[Authorize]` on `BaseController`) plus per-resource ownership rules enforced in the services (edit/delete own tools, return loans of own tools, etc.).

11. Configuration and Operations

11.1 Configuration files

File	Contents
`appsettings.json`	Connection string `DefaultConnection` (LocalDB), logging, `AllowedHosts`, and the production Identity password policy
`appsettings.Development.json`	Connection string, logging and a relaxed Identity password policy for development
`Properties/launchSettings.json`	Profiles `http` (http://localhost:5140) and `https` (https://localhost:7063; http://localhost:5140); both set ASPNETCORE_ENVIRONMENT=Development
`CommuniLend.Web.csproj`	net10.0, nullable and implicit usings, UserSecretsId, package references

11.2 Identity password options

Option	appsettings.json (production)	appsettings.Development.json
RequiredLength	8	3
RequireDigit	true	false
RequireLowercase	true	false
RequireUppercase	true	false
RequireNonAlphanumeric	false	false
RequiredUniqueChars	4	1

Configuration sources apply in this order (later wins): appsettings.json, appsettings.{Environment}.json, user secrets (Development only), environment variables. To point the app to another SQL Server without editing files:

```
cd CommuniLend.Web
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Server=localhost;Database=CommuniLend;User Id=sa;Pass
```

Alternatively set the environment variable `ConnectionStrings__DefaultConnection`.

11.3 Build, migrate, run

```
dotnet restore
dotnet ef database update --project CommuniLend.Data --startup-project CommuniLend.Web
dotnet run --project CommuniLend.Web --launch-profile https
```

The app is then available at `https://localhost:7063` (or `http://localhost:5140`). Seeded sign-in: any seeded e-mail with password **Password123!** (see chapter 12).

11.4 Static assets and uploads

Tool photos are served from `wwwroot/images/tools/`, including a `no-image.jpg` placeholder. The seed images are also shipped as `CommuniLend.Web/seed-images/tools.rar` (included as project content). Client libraries (Bootstrap, jQuery, jQuery Validation) are bundled in `wwwroot/lib`.

12. Seed Data Reference

Reference date for relative offsets: **28 Sep 2026**. All users: password **Password123!**.

12.1 Members

#	Name	E-mail	Phone	Joined
1	John Smith	john.smith@email.com	(555) 123-4567	30 Jun 2026
2	Sarah Johnson	sarah.johnson@email.com	(555) 234-5678	30 Jul 2026
3	Michael Chen	michael.chen@email.com	(555) 345-6789	14 Aug 2026
4	Emily Rodriguez	emily.rodriguez@email.com	(555) 456-7890	29 Aug 2026
5	David Williams	david.williams@email.com	(555) 567-8901	08 Sep 2026
6	Jessica Martinez	jessica.martinez@email.com	(555) 678-9012	13 Sep 2026
7	Robert Taylor	robert.taylor@email.com	(555) 789-0123	18 Sep 2026

12.2 Categories

Id	Name	Description	Tools
1	Power Tools	Electric and cordless power tools for drilling, cutting, sanding, and more	17
2	Hand Tools	Manual hand tools including wrenches, hammers, screwdrivers, pliers, and socket sets	19
3	Measuring & Finishing Tools	Tools for measuring, leveling, detecting studs, and finishing work	14

12.3 Open loans (7)

Id	Tool	Lender	Borrower	Loan date	Due date	Notes
1	Adjustable Wrench Set	Sarah Johnson	John Smith	14 Sep 2026	05 Oct 2026	Borrowed for home renovation project
2	Tape Measure	Jessica Martinez	Sarah Johnson	18 Sep 2026	09 Oct 2026	Needed for measuring deck
3	Screwdriver Set	Michael Chen	Robert Taylor	20 Sep 2026	11 Oct 2026	Cabinet installation
4	Flashlight	Jessica Martinez	Emily Rodriguez	23 Sep 2026	14 Oct 2026	Basement repair work
5	Caulking Gun	Emily Rodriguez	David Williams	25 Sep 2026	16 Oct 2026	Caulking bathroom tiles
6	Socket Set	Robert Taylor	Jessica Martinez	26 Sep 2026	17 Oct 2026	Engine repair project
7	Laser Measure	Robert Taylor	Michael Chen	27 Sep 2026	18 Oct 2026	Measuring for new flooring

12.4 Closed loans (9)

Id	Tool	Lender	Borrower	Loan	Due	Returned	Notes
8	Level	Michael Chen	Sarah Johnson	19 Aug 2026	02 Sep 2026	03 Sep 2026	Shelf installation
9	Pliers Set	John Smith	Michael Chen	19 Aug 2026	02 Sep 2026	03 Sep 2026	Picture frame hanging
10	Hammer Set	Emily Rodriguez	John Smith	31 Aug 2026	14 Sep 2026	15 Sep 2026	Fence repair
11	Stud Finder	David Williams	Emily Rodriguez	10 Sep 2026	24 Sep 2026	25 Sep 2026	Wall stud detection
12	Circular Saw	John Smith	Michael Chen	04 Sep 2026	18 Sep 2026	19 Sep 2026	Deck board cutting
13	Miter Saw	Sarah Johnson	David Williams	10 Sep 2026	24 Sep 2026	25 Sep 2026	Molding and trim work
14	Reciprocating Saw	John Smith	Sarah Johnson	24 Aug 2026	07 Sep 2026	08 Sep 2026	Tree removal branch cutting
15	Orbital Sander	Sarah Johnson	John Smith	31 Aug 2026	14 Sep 2026	15 Sep 2026	Floor sanding
16	Nail Set	Michael Chen	Emily Rodriguez	06 Sep 2026	20 Sep 2026	21 Sep 2026	Finishing hardwood

12.5 Tools (50)

Id	Name	Cat.	Own.	Cond.	EUR	Added	Image
1	Cordless Drill	Power	1	New	89.99	29 Aug 2026	cordless-drill.jpg
2	Circular Saw	Power	1	Good	65.50	03 Sep 2026	circular-saw.jpg
3	Adjustable Wrench Set	Hand	2	Good	25.99	08 Sep 2026	wrench-set.jpg
4	Level	Meas.	3	Fair	18.75	16 Aug 2026	level.jpg
5	Hammer Set	Hand	4	Good	35.00	30 Aug 2026	hammer-set.jpg
6	Jigsaw	Power	5	New	120.00	20 Sep 2026	jigsaw.jpg
7	Tape Measure	Meas.	6	Good	12.50	14 Sep 2026	tape-measure.jpg
8	Socket Set	Hand	7	Fair	45.99	23 Sep 2026	socket-set.jpg
9	Pliers Set	Hand	1	Good	28.00	30 Jul 2026	pliers-set.jpg
10	Miter Saw	Power	2	New	199.99	09 Aug 2026	miter-saw.jpg
11	Screwdriver Set	Hand	3	Good	19.99	16 Sep 2026	screwdriver-set.jpg
12	Power Drill	Power	4	Fair	59.99	28 Sep 2026	power-drill.jpg
13	Stud Finder	Meas.	5	New	42.50	09 Sep 2026	stud-finder.jpg
14	Flashlight	Meas.	6	Good	15.00	18 Sep 2026	flashlight.jpg
15	Laser Measure	Meas.	7	New	79.99	22 Sep 2026	laser-measure.jpg
16	Reciprocating Saw	Power	1	Good	145.00	09 Aug 2026	reciprocating-saw.jpg
17	Orbital Sander	Power	2	Good	99.99	19 Aug 2026	orbital-sander.jpg
18	Nail Set	Hand	3	Worn	8.99	24 Aug 2026	nail-set.jpg
19	Caulking Gun	Hand	4	Good	22.00	20 Sep 2026	caulking-gun.jpg
20	Utility Knife	Hand	5	Fair	10.50	28 Sep 2026	utility-knife.jpg
21	Impact Driver	Power	6	New	99.99	28 Sep 2026	impact-driver.jpg
22	Brad Nailer	Power	7	New	75.00	28 Sep 2026	brad-nailer.jpg
23	Stud Finder Kit	Meas.	1	Good	35.99	28 Sep 2026	stud-kit.jpg
24	Torpedo Level	Meas.	2	Good	16.50	28 Sep 2026	torpedo-level.jpg
25	Ratchet Set	Hand	3	Good	55.00	28 Sep 2026	ratchet-set.jpg
26	Bench Vise	Hand	4	Fair	65.00	28 Sep 2026	bench-vise.jpg
27	Angle Grinder	Power	5	Good	85.99	28 Sep 2026	angle-grinder.jpg
28	Pneumatic Nailer	Power	6	Fair	60.00	28 Sep 2026	pneumatic-nailer.jpg
29	Magnetic Level	Meas.	7	New	21.99	28 Sep 2026	magnetic-level.jpg
30	Power Saw	Power	1	Worn	55.00	28 Sep 2026	power-saw.jpg
31	Allen Key Set	Hand	2	Good	14.99	28 Sep 2026	allen-key-set.jpg
32	Pipe Wrench	Hand	3	Fair	32.50	28 Sep 2026	pipe-wrench.jpg
33	Hacksaw	Hand	4	Good	13.00	28 Sep 2026	hacksaw.jpg
34	Drill Bit Set	Hand	5	Good	18.50	28 Sep 2026	drill-bit-set.jpg
35	Combination Square	Meas.	6	New	19.99	28 Sep 2026	combination-square.jpg
36	Flashlight Set	Meas.	7	Good	35.00	28 Sep 2026	flashlight-set.jpg
37	Pry Bar	Hand	1	Good	14.50	28 Sep 2026	pry-bar.jpg

Id	Name	Cat.	Own.	Cond.	EUR	Added	Image
38	Oscillating Multi-Tool	Power	2	New	129.99	28 Sep 2026	multi-tool.jpg
39	Bar Clamp Set	Hand	3	Fair	42.00	28 Sep 2026	bar-clamp-set.jpg
40	Nail Gun	Power	4	Good	79.99	28 Sep 2026	nail-gun.jpg
41	Carpenter's Square	Meas.	5	Worn	11.00	28 Sep 2026	carpenters-square.jpg
42	Dust Extractor	Power	6	Good	125.00	28 Sep 2026	dust-extractor.jpg
43	Adjustable Square	Meas.	7	New	17.50	28 Sep 2026	adjustable-square.jpg
44	Cable Protector	Hand	1	Good	28.00	28 Sep 2026	cable-protector.jpg
45	Bench Grinder	Power	2	Fair	95.00	28 Sep 2026	bench-grinder.jpg
46	Chisel Set	Hand	3	Good	24.50	28 Sep 2026	chisel-set.jpg
47	Retractable Measuring Wheel	Meas.	4	New	35.99	28 Sep 2026	measuring-wheel.jpg
48	Sheet Metal Shears	Hand	5	Fair	33.00	28 Sep 2026	sheet-metal-shears.jpg
49	Angle Finder	Meas.	6	New	38.50	28 Sep 2026	angle-finder.jpg
50	Rotary Tool Kit	Power	7	Good	89.99	28 Sep 2026	rotary-tool-kit.jpg

Cat.: Power = Power Tools, Hand = Hand Tools, Meas. = Measuring & Finishing Tools. Owner is the member number from 12.1.

13. Observations on Current Behavior

The points below were found while reading the code. They are not necessarily defects; they are listed so that the behavior is documented and can be reviewed deliberately. Three earlier observations (password minimum on the Register page, e-mail change not updating the login name, and replaced photos not being deleted) have been fixed in the code and are reflected in the chapters above.

#	Area	Observation
1	Upload limit on Edit	Only Add carries `[RequestSizeLimit]`/`[RequestFormLimits]`; the 50 KB limit on Edit is enforced by the view-model attribute.
2	Loan start date	A loan must start in the future, but the tool is treated as On Loan as soon as the open loan exists (status depends only on the open loan and due date).
3	Return result	`LoansController.Return` ignores the boolean result and always redirects to the loan list.
4	Time zone	“Today” is always the UTC date (status, return date, date added, loan validation); members in other time zones may see off-by-one-day effects around midnight.
5	Tool history	Deleting a tool deletes its closed-loan history; deleting an account deletes closed loans in which the member was lender or borrower, which also removes the counterpart's history entries.
6	Tool details access	Any signed-in member can open `/Tools/Details/{id}` for any tool, including tools of inactive members (shown as Locked).
7	Image names	Stored file names derive from the client-supplied file name (path stripped by `Path.GetFileName`); collisions are resolved with a GUID prefix. Content is validated by extension, size and file signature.

Appendix A. Source File Inventory

Project	Key files
CommuniLend.Data.Models	ApplicationUser.cs, Tool.cs, Category.cs, OpenLoan.cs, ClosedLoan.cs, Enums/ToolCondition.cs
CommuniLend.DataModelConstraints	ValidationConstants.cs, ColumnTypes.cs
CommuniLend.Data	CommuniLendDbContext.cs; Configurations/ (ApplicationUser, Category, Tool, OpenLoan, ClosedLoan, StaticSeedData/SeedUserIds, ReferenceSeedDate); Migrations/ (4 migrations + snapshot)
CommuniLend.Services	Contracts/ (IToolsService, ILoansService, IUsersService, IImageStorageService); ToolsService, LoansService, UsersService, LocalImageStorageService; Extensions/ (DateOnlyExtensions, ToolQueryExtensions)
CommuniLend.Web.ViewModels	Category/, Enums/, Loan/, Tool/, User/
CommuniLend.Web	Program.cs; Controllers/ (BaseController, Tools, Loans, Users); Views/; Areas/Identity/Pages/Account/ (+ RegistrationToken.cs); wwwroot/; appsettings*.json; Properties/launchSettings.json; seed-images/tools.rar